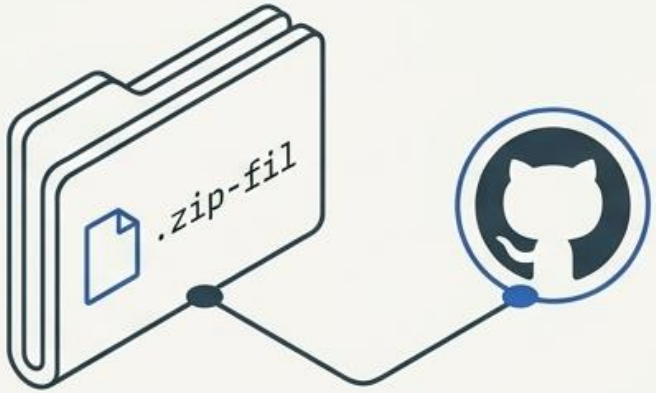
Projektförberedelse & Övning

Arseniy Buzyan

Koden



Inlämning sker via `.zip-fil`. `README.md` måste innehålla en aktiv länk till ditt GitHub-repository.

Försvaret



Muntlig redovisning (max 10 minuter inklusive frågor). Det viktigaste är inte att koden är perfekt, utan att du kan förklara din kod och visa förståelse för flödet.

Examinationen – Vad Krävs För G?



Individuellt projekt + muntlig redovisning (max 10 min)



G-krav (3 obligatoriska filer):



PowerShell-skript (.ps1) Fungerande skript med parametrar och felhantering



Terraform-konfiguration (.tf) Fungerande HCL-kod med variabler och outputs



README (.md) Mål, metod, resultat, reflektion + GitHub-länk, 5+ commits

Tekniska krav för G

Område	Vad krävs?
PowerShell	Funktioner, loopar, villkor, felhantering (try/catch)
Terraform	Provider (local), minst 2 resources, variabler, outputs
Kedja	Terraform → PowerShell (minst 2 verktyg samverkar)
Verifiering	PowerShell verifierar resultatet
GitHub	Repository med minst 5 commits

VG-krav (extra 2 filer)

VG-tekniska krav

För VG ska du visa högre kvalitet i måluppfyllelsen – t.ex. större skicklighet, säkrare utförande eller bättre analys . Det handlar om att du inte bara får det att fungera, utan att du gör det på ett genomtänkt och professionellt sätt. Se till att loggning finns i ALLA skript.

Fil	Krav
Python-skript (.py)	Verifiering eller orkestrering
Ansible-playbook (.yml)	Fungerande playbook som konfigurerar system

Område	Vad krävs?
Optimering	Parametrar, felhantering, loggning i alla skript
Kedja	Komplett kedja: Terraform → Ansible → PowerShell → Python
GitHub	Minst 10 commits med tydliga meddelanden
README	7 avsnitt + installationsinstruktioner

Exempel – Komplett Projekt

Detta är ett exempel på hur ett komplett G-projekt kan se ut.

Exakt så här bör din filstruktur se ut innan du skapar din .zip-fil.

```
/mitt-automationsprojekt
├── [VG] playbook.yml     # Konfigurerar systemet
├── [VG] orchestrate.py   # Verifiering eller orkestrering
├── main.tf               # Minst 2 resources, variabler, outputs
├── verify.ps1            # Funktioner, loopar, villkor, try/catch
└── README.md             # Dokumentation & GitHub-länk
```

Steg 1: Provisionera (Terraform)

```
resource "local_file" "config" {
    filename = "config.json"
    content  = jsonencode({
      server_name = var.server_name
      port        = var.port
    })
}
```

Provider (local): Skapar konfigurationsfilen lokalt på disken.

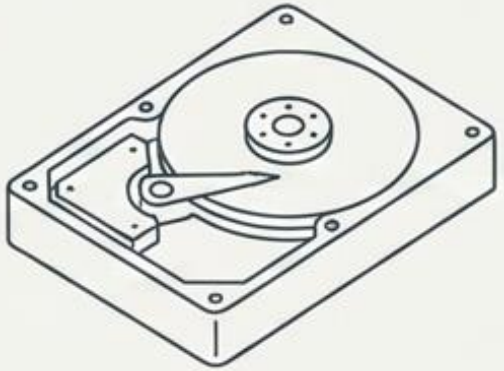
Variabler & Outputs: Dynamisk data injiceras in i JSON-formatet för nästa steg.

Steg 1 – Terraform (provisionera)

`local_file` är en resurs från Terraform local provider som hanterar filer på din lokala dator . Den är perfekt för att skapa konfigurationsfiler som andra verktyg sedan kan använda i automationskedjan.

```hcl
# main.tf
resource "local_file" "config" {
  filename = "C:/Projekt/Config/config.json"
  content = jsonencode({
    server_name = var.server_name
    port = var.port
    environment = var.environment
  })
}
```

```
port = 8080
```



config.json:
{"port": 8080}

```
ConvertFrom-Json
→ $config.port
```

Handoff-mekanismen
Verktygen pratar inte direkt med varandra. Terraform exporterar variabler till en fysisk JSON-fil, som PowerShell sedan läser in för att fortsätta kedjan.

Steg 2: Konfigurera & Verifiera (PowerShell)



Ingestion: Importerar inställningarna från Terraforms output.

Aktion: Utför den faktiska systemförändringen (funktioner, loopar, villkor).

Verifiering: Kontrollerar resultatet av automationskedjan.

Steg 2 – PowerShell (konfigurera och verifiera)

```powershell
# verify.ps1
$config = Get-Content "C:/Projekt/Config/config.json" | ConvertFrom-Json

try {
    Install-WindowsFeature -Name Web-Server
    Start-Service -Name W3SVC
    $response = Invoke-WebRequest -Uri "http://localhost:$($config.port)"
    Write-Host "✅ Webbserver svarar!" -ForegroundColor Green
} catch {
    Write-Host "❌ Fel vid installation/verifiering" -ForegroundColor Red
}
```

Steg 3 – README

- Mål: Automatisera installation av webbserver med IaC
- Metod: Terraform → PowerShell
- Reflektion: Vad gick bra? Vad var svårt?
- **Koppling till kursen:** Detta är exakt vad ni har övat på i tidigare teman (PowerShell, Terraform, verifiering). Här används Terraform för att skapa `config.json`, och PowerShell läser denna JSON för att veta vad som ska installeras.

Felsökning av Vanliga Problem

Här är de vanligaste problemen och hur du löser dem.

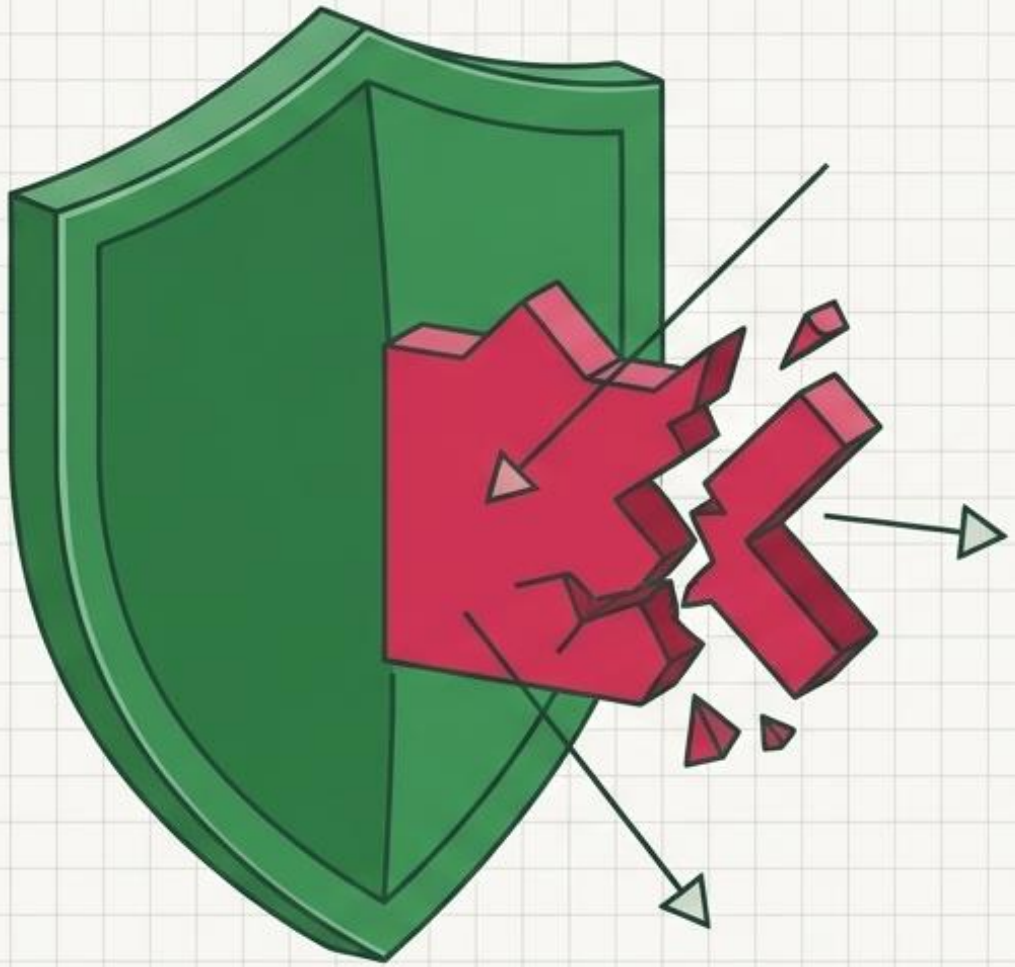
Problem	Orsak	Lösning
Terraform hittar inte provider	Glömt terraform init	Kör terraform init först
PowerShell skriptet kraschar	Saknar felhantering	Lägg till try/catch
Ansible kan inte ansluta till Windows	WinRM inte aktiverat	Kör Enable-PSRemoting -Force
JSON-filen hittas inte	Fel sökväg	Använd absolut sökväg
Git push misslyckas	Ingen remote repository	Skapa repo på GitHub först
Permission denied	Inte tillräckliga rättigheter	Kör PowerShell som Administrator

Allmän felsökningsteknik

Proffstips: Om du kör fast, testa varje del separat innan du försöker köra hela kedjan! Felsökning handlar om att isolera problemet – hitta var det går fel och fokusera på den delen.

Teknik	Varför?
Kontrollera sökvägar	Absoluta sökvägar är säkrare än relativa
Lägg till loggning	Utan loggning vet du inte vad som hände
Testa steg för steg	Terraform för sig, PowerShell för sig – sedan ihop
Kontrollera behörigheter	Kör PowerShell som Administrator vid behov

Try/Catch-Skölden



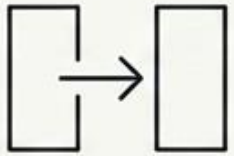
Kravet: Ett skript som kraschar är ett underkänt skript. För betyget G krävs aktiv felhantering.

Mekaniken:

- `try { ... }`: Försök köra koden här inne.
- `catch { ... }`: Om något går fel, fånga felet här och stäng ner skriptet snyggt med ett felmeddelande i röd text.

För VG: Denna sköld (och loggning) måste finnas i samtliga skript, inte bara PowerShell.

Allmän Felsökningsteknik



Isolering

Testa steg för steg. Kör `Terraform` för sig, sedan `PowerShell` för sig. Sätt ihop dem först när båda fungerar.



Spårbarhet

Lägg till loggning. Utan loggar arbetar du i blindo när kedjan växer.



Precision

Absoluta sökvägar är alltid säkrare än relativa i automationskedjor.



Auktorisation

Kontrollera behörigheter. Kör `PowerShell` som Administratör vid systemförändringar.

- Här är svar på de vanligaste frågorna om examinationen.
- Kom ihåg: Projektet bedöms som en helhet. Brister inom vissa områden kan kompenseras av styrkor inom andra! Enligt MYH sätts betyg utifrån resultatet av flera kunskapskontroller, där helheten bedöms.

Vanliga Frågor (FAQ) – Examinationen

Fråga	Svar
Hur lång ska redovisningen vara?	Max 10 minuter inklusive frågor
Måste jag använda Ansible?	Endast för VG
Måste jag använda Python?	Endast för VG
Får jag använda AI-verktyg?	Ja, men du måste förstå ALL kod du lämnar in
Vad är viktigast under redovisningen?	Att du kan förklara din kod och visa förståelse
Hur lämnar jag in?	.zip-fil + GitHub-länk i README
Vad händer om jag misslyckas?	Du har rätt till två omexaminationer

VG-Kedjan: Komplett Orkestrering

1. Terraform

Provisionerar & Exporterar variabler

2. Ansible

Konfigurerar operativsystemet (Windows/Linux)

3. PowerShell

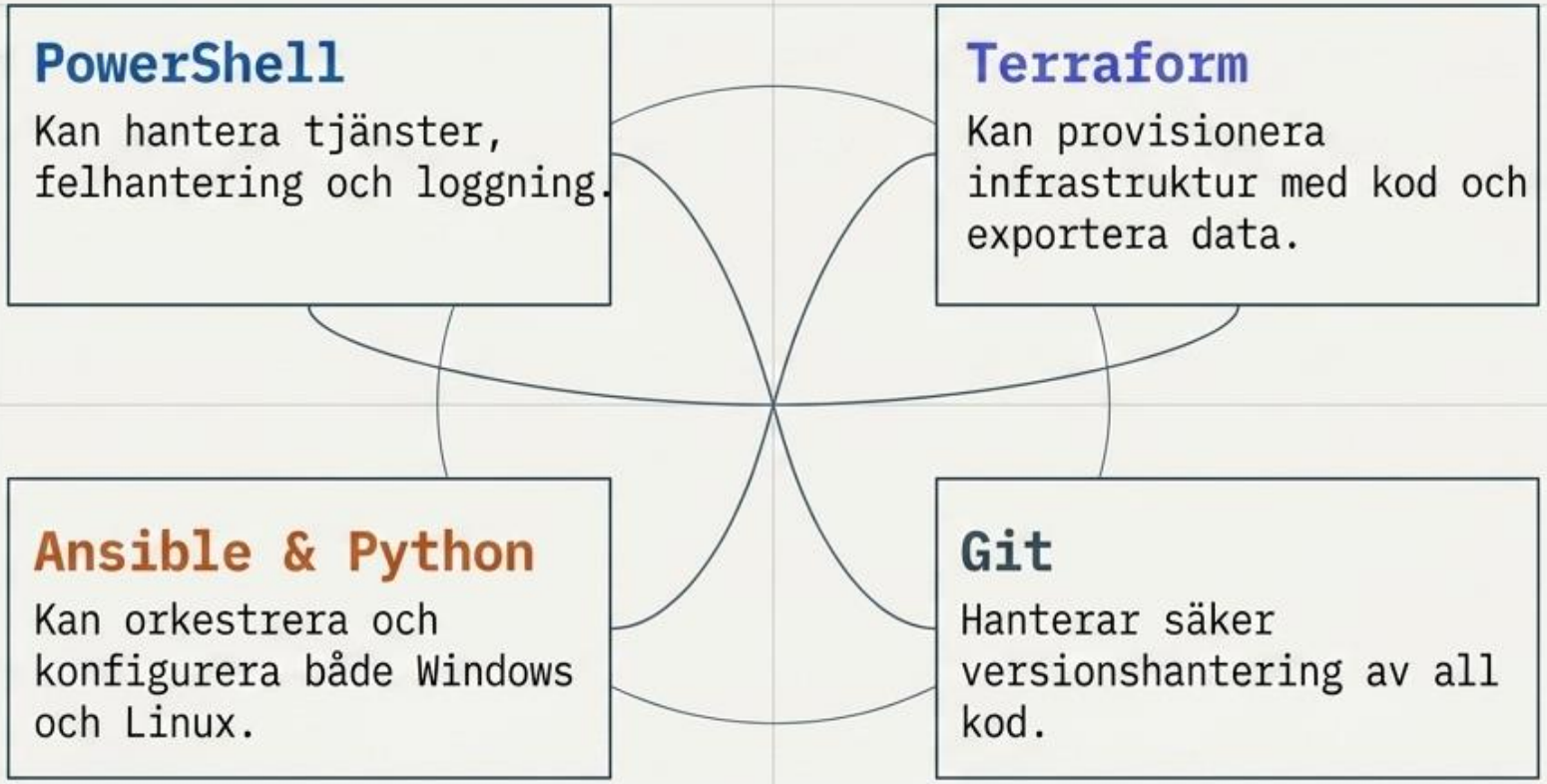
Verkställer specifika systemtjänster

4. Python

Orkestrerar flödet och verifierar slutmålet

För VG räcker det inte att verktygen finns; de måste samverka i en obruten, loggad och felhanterad kedja.

Din Nya Automationsprofil



Detta är inte bara en inlämning. Det är din nya kompetensprofil som IT-proffs. Använd denna insikt under din redovisning. Lycka till!

Vad du nu kan

Detta är din kompetensprofil – använd den i din redovisning!

Verktyg	Vad du kan göra
PowerShell	Skriva skript, hantera tjänster, felhantering, loggning
Terraform	Provisionera infrastruktur med kod (IaC) – inklusive local provider för att skapa filer och JSON
Ansible	Konfigurera servrar, både Linux och Windows
Python	Orkestrera och verifiera
Git	Versionshantering av all kod

Quiz

https://forms.gle/7a1Qta3eLTidtPBX7